SISTEMAS DISTRIBUIDOS

Fundamentos, Implementação e Experimentação

Apresentação

Esta apostila foi concebida como um guia de estudo e laboratório para uma disciplina de Sistemas Distribuídos em nível de graduação. O objetivo não é tratar o tema apenas como um catálogo de tecnologias, mas construir um percurso em que os mecanismos concretos de implementação sejam continuamente relacionados aos modelos e problemas fundamentais da área: comunicação, concorrência, ausência de memória global, falhas parciais, coordenação, segurança, replicação e descentralização.

A proposta didática combina explicações conceituais, exemplos executáveis, experimentos controlados e atividades práticas. Em vez de apresentar REST, MQTT e RPC como assuntos isolados, a apostila os utiliza como diferentes respostas para uma mesma pergunta: como componentes autônomos, executados em processos ou máquinas distintas, podem cooperar para formar um sistema coerente? Na segunda metade, os mesmos componentes são submetidos a atrasos, quedas, perda de conectividade e ameaças, preparando o estudante para compreender por que sistemas distribuídos reais precisam ser projetados explicitamente para operar sob incerteza.

Como estudar com esta apostila
Em cada capítulo, leia os fundamentos antes de executar o laboratório. Durante os experimentos, registre hipóteses e evidências. O comportamento observado deve ser explicado com vocabulário técnico: latência, timeout, idempotência, acoplamento, disponibilidade, falha por omissão, consistência, replicação, entre outros. O objetivo não é apenas “fazer funcionar”, mas compreender por que funciona e como falha.

Organização pedagógica

Tabela 1 - Percurso de aprendizagem da apostila

Bloco	Questão central	Tecnologias / conceitos
I. Fundamentos	O que torna um sistema distribuído diferente de um programa local?	Arquiteturas, transparência, escalabilidade, heterogeneidade
II. Comunicação	Como processos trocam informações?	REST/HTTP, MQTT, gRPC/Protocol Buffers
III. Engenharia de falhas	O que acontece quando rede ou processos deixam de se comportar como esperado?	Interação, falhas, segurança, timeouts, retries
IV. Descentralização	Como manter estado e confiança sem um coordenador único?	Blockchain, hashing, consenso, replicação
V. Programabilidade distribuída	Como executar regras sobre um ledger compartilhado?	EVM, Solidity, smart contracts
VI. Integração	Como combinar mecanismos em uma arquitetura coerente?	Trabalho Prático Integrador

1. Introdução aos Sistemas Distribuídos

Objetivos de aprendizagem

- Definir sistema distribuído e distinguir distribuição de simples comunicação em rede.
- Explicar os principais desafios introduzidos por concorrência, ausência de relógio global e falhas independentes.
- Reconhecer transparência, escalabilidade, heterogeneidade e abertura como propriedades de projeto.
- Analisar exemplos cotidianos e identificar seus componentes distribuídos.

1.1 O que é um sistema distribuído?

Um sistema distribuído é formado por componentes computacionais autônomos que se comunicam por uma rede e coordenam suas ações para oferecer um serviço ou comportamento percebido como uma unidade lógica. A palavra autônomo é importante: cada processo possui seu próprio fluxo de execução e sua própria memória; não existe uma memória compartilhada global nem um mecanismo mágico que faça todos os componentes observarem os mesmos eventos ao mesmo tempo.

A distribuição surge quando partes relevantes da aplicação estão separadas por fronteiras de processo, máquina, contêiner, dispositivo ou domínio administrativo. Uma aplicação Web simples já pode ser distribuída quando o navegador executa em um computador, a API em outro e o banco de dados em um terceiro. A complexidade aumenta quando adicionamos caches, filas de mensagens, réplicas, balanceadores, serviços externos e componentes executados na borda.

Ideia-chave
Em um programa local, uma chamada de função ou acesso à memória costuma falhar de forma evidente. Em um sistema distribuído, uma chamada remota pode demorar, ser duplicada, chegar depois de um retry, alcançar um servidor parcialmente indisponível ou produzir uma resposta que se perde no caminho. A rede transforma incerteza em parte do modelo de programação.

1.2 Três propriedades que mudam tudo

Concorrência

Vários componentes executam simultaneamente e podem disputar recursos. Como não há uma única linha de execução, o resultado pode depender da ordem relativa dos eventos. Atualizações concorrentes sobre o mesmo dado podem gerar condições de corrida, perda de atualização ou necessidade de coordenação.

Ausência de relógio global

Cada máquina possui relógio próprio, sujeito a desvio e ajustes. Mesmo quando relógios são sincronizados por protocolos como NTP, não se deve assumir igualdade perfeita. Por isso, ordenação causal e relógios lógicos são conceitos importantes: frequentemente é mais útil saber que um evento ocorreu antes de outro por causalidade do que comparar timestamps físicos com precisão absoluta.

Falhas independentes e parciais

Um componente pode falhar enquanto outros continuam executando. Um cliente pode estar ativo, o servidor pode estar ativo, mas a rota de rede entre eles pode estar indisponível. Essa falha parcial é uma característica definidora de sistemas distribuídos: observar um silêncio não permite distinguir imediatamente entre servidor lento, servidor falho, rede congestionada ou perda de resposta.

1.3 Objetivos de projeto

Tabela 2 - Objetivos comuns em sistemas distribuídos

Objetivo	Significado prático	Risco se ignorado
Compartilhamento de recursos	Dados, serviços, sensores e capacidade computacional podem ser acessados remotamente.	Ilhas de informação e duplicação desnecessária.
Transparência	Ocultar seletivamente detalhes de localização, replicação, migração ou acesso.	Complexidade excessiva para usuários e desenvolvedores.
Escalabilidade	Crescer em usuários, dados, máquinas e regiões sem degradação descontrolada.	Gargalos, custos explosivos e indisponibilidade.
Heterogeneidade	Integrar diferentes sistemas operacionais, linguagens, redes e dispositivos.	Acoplamento a uma única plataforma.
Abertura	Usar interfaces e protocolos documentados, permitindo substituição e interoperabilidade.	Aprisionamento tecnológico e evolução difícil.
Segurança	Preservar confidencialidade, integridade, autenticidade e disponibilidade.	Exposição de dados e controle indevido.

1.4 Transparência não significa invisibilidade total

Transparência é a capacidade de esconder detalhes de distribuição quando isso simplifica o uso, mas abstrações perfeitas podem ser perigosas. Uma chamada remota não deve ser tratada exatamente como uma chamada local porque seus custos e modos de falha são diferentes. Uma boa API pode esconder detalhes de serialização, mas ainda precisa tornar explícitos timeouts, erros de rede e semântica de retries.

- Transparência de acesso: usar operações semelhantes para recursos locais e remotos.
- Transparência de localização: não exigir que o usuário conheça o endereço físico do recurso.
- Transparência de replicação: apresentar várias cópias como uma unidade lógica.
- Transparência de concorrência: permitir acesso simultâneo preservando propriedades definidas.
- Transparência de falhas: mascarar algumas falhas sem esconder situações que exigem decisão da aplicação.

1.5 Escalabilidade: tamanho, geografia e administração

Escalar não significa apenas adicionar servidores. Um sistema pode precisar crescer em três dimensões. Na escala de tamanho, aumenta o número de usuários ou componentes; na escala geográfica, crescem as distâncias e latências; na escala administrativa, diferentes organizações ou equipes passam a operar partes do sistema. Cada dimensão introduz desafios próprios, como particionamento de dados, cache, replicação, observabilidade e contratos de interface.

1.6 Exemplos reais analisados como sistemas distribuídos

Tabela 3 - Leitura distribuída de sistemas conhecidos

Sistema	Componentes distribuídos	Problema dominante
Aplicação de comércio eletrônico	Frontend, API, catálogo, pagamentos, estoque, banco, cache	Consistência entre pedido, pagamento e estoque
Streaming de vídeo	CDNs, origem, autenticação, recomendação, métricas	Latência, escala geográfica e cache

Sistema	Componentes distribuídos	Problema dominante
IoT industrial	Sensores, gateways, broker, serviços, banco, dashboards	Conectividade intermitente e dados em fluxo
Sistema bancário	Canais digitais, autorização, ledger, antifraude, mensageria	Integridade, atomicidade e auditoria
Blockchain pública	Nós P2P, mempool, consenso, ledger replicado	Acordo entre participantes não confiáveis

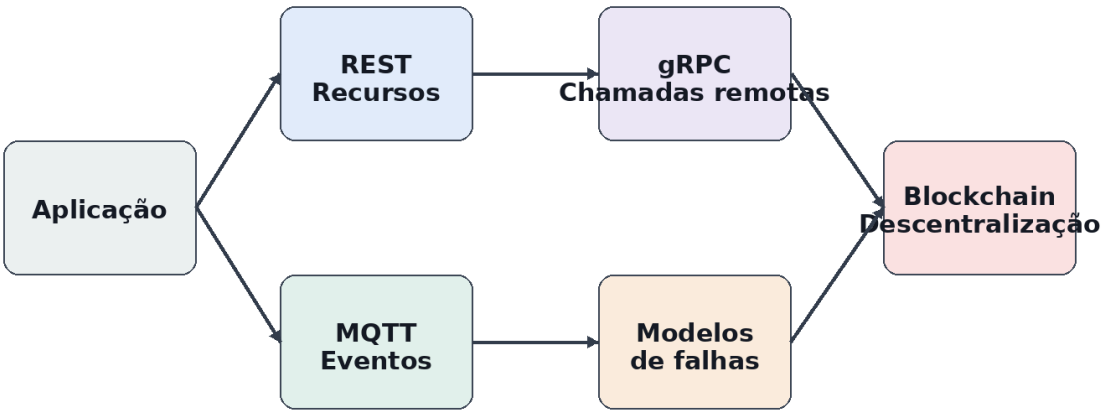


Figura 2 - Progressão conceitual: mecanismos de comunicação, falhas e descentralização

Exercícios de fixação

1. Explique por que um sistema com dois processos na mesma máquina pode ser considerado distribuído em alguns contextos, mesmo sem duas máquinas físicas.

2. Diferencie falha total de falha parcial e dê um exemplo de cada uma.

3. Uma equipe decide esconder todos os erros de rede do cliente e repetir requisições indefinidamente. Qual transparência está sendo buscada e quais riscos surgem?

4. Identifique pelo menos cinco componentes distribuídos em um aplicativo de transporte por demanda e descreva uma interação entre dois deles.

5. Explique por que “a rede é rápida” não é uma propriedade na qual uma arquitetura distribuída robusta deve se apoiar.

1.7 As falácias clássicas da computação distribuída

Uma forma útil de amadurecer o raciocínio arquitetural é explicitar suposições que parecem convenientes durante o desenvolvimento, mas falham em produção. As chamadas “falácias da computação distribuída” lembram que a rede não é perfeitamente confiável, a latência não é zero, a largura de banda não é infinita, a topologia muda, existem múltiplos administradores, há custos de transporte e a rede não é homogênea nem automaticamente segura. Elas não são leis formais; são alertas de engenharia contra premissas implícitas.

O efeito mais importante dessas falácias é obrigar o projeto a tornar explícitos limites e mecanismos de recuperação. Se a rede pode falhar, precisamos de timeout. Se mensagens podem ser duplicadas, precisamos de idempotência ou deduplicação. Se a topologia muda, precisamos de descoberta e

configuração. Se existem administradores distintos, contratos, observabilidade e políticas de segurança precisam atravessar fronteiras organizacionais.

Tabela 4 - Falácias comuns e respostas de engenharia

Suposição ingênua	Consequência provável	Resposta de projeto
A rede é confiável	Chamadas bloqueiam ou falham sem tratamento	Timeout, retry seletivo, circuit breaker
A latência é zero	Fluxos chatty multiplicam tempo de resposta	Batch, cache, menos hops, proximidade
A banda é infinita	Payloads grandes saturam links e filas	Compactação, compressão, paginação, streaming
A topologia não muda	Endereços fixos quebram com escala e failover	Descoberta, DNS, balanceamento
A rede é segura	Credenciais e dados ficam expostos	TLS, autenticação, autorização, segmentação
Existe um administrador	Políticas divergem entre equipes/organizações	Contratos, governança, SLOs, versionamento

1.8 Estado local, estado compartilhado e fronteiras de consistência

Uma decisão central é onde o estado vive. Estado local é simples e rápido, mas desaparece quando o processo reinicia e não é automaticamente visível a outras réplicas. Estado compartilhado em banco ou serviço externo facilita coordenação, porém cria dependência de rede e pode se tornar gargalo. Replicar estado melhora disponibilidade e leitura, mas introduz sincronização e consistência. Assim, “tornar o servidor stateless” normalmente significa mover o estado de sessão para um mecanismo compartilhado ou para o próprio cliente, e não eliminar todo estado do sistema.

Ao analisar um componente, pergunte: que informação ele precisa lembrar? Essa informação precisa sobreviver a reinício? Outros nós precisam vê-la imediatamente? Pode existir atraso de propagação? A resposta determina se o estado pode ficar em memória, precisa de armazenamento durável ou exige replicação com garantias mais fortes.

1.9 Latência em cadeia e orçamento de tempo

Uma requisição externa pode atravessar gateway, serviço A, serviço B e banco. Se cada hop tiver latência modesta, a soma pode ultrapassar o objetivo total. Além disso, caudas de latência se acumulam: quando um serviço depende de várias chamadas paralelas, a resposta final tende a esperar o componente mais lento. Por isso, sistemas distribuídos trabalham com orçamento de latência e deadlines propagados, e não com timeouts escolhidos isoladamente.

Pergunta de projeto
Qual é o tempo máximo útil da operação para o usuário ou processo de negócio? A partir desse limite, distribua orçamento entre hops e reserve margem para rede, fila, processamento e retries. Um timeout maior que a utilidade da resposta apenas prolonga sofrimento.

2. Arquiteturas de Sistemas Distribuídos

Objetivos de aprendizagem

- Comparar arquiteturas cliente-servidor, multicamadas, peer-to-peer, orientadas a serviços e orientadas a eventos.
- Distinguir arquitetura lógica de topologia física de implantação.
- Avaliar acoplamento, escalabilidade, resiliência e complexidade operacional.
- Selecionar uma arquitetura coerente com requisitos funcionais e não funcionais.

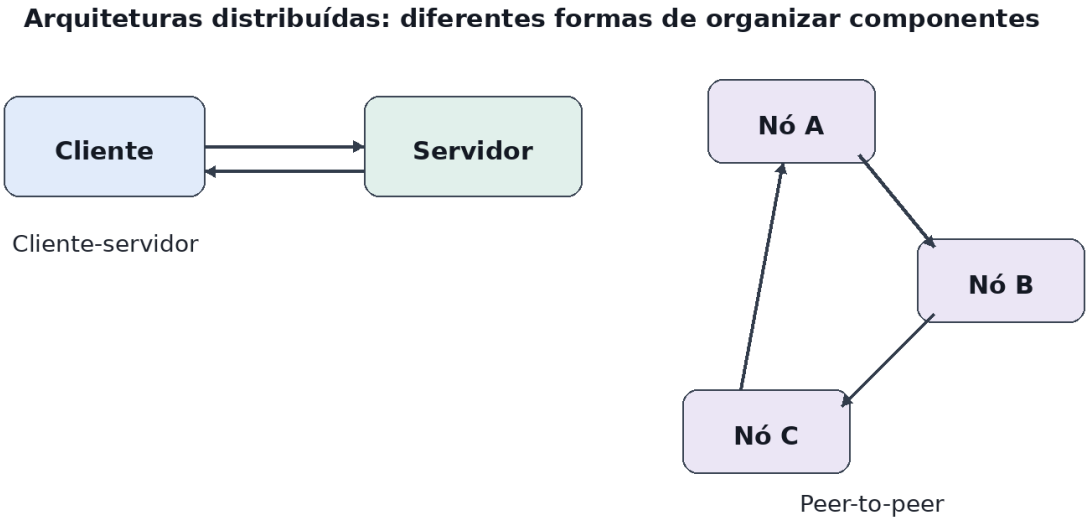


Figura 3 - Contraste simplificado entre cliente-servidor e peer-to-peer

2.1 Arquitetura lógica versus implantação

Arquitetura lógica descreve responsabilidades e relações entre componentes: quem oferece serviço, quem consome, onde reside o estado e como ocorre a comunicação. Implantação descreve onde esses componentes executam fisicamente ou virtualmente. Um sistema logicamente dividido em três camadas pode ser implantado em uma única máquina durante desenvolvimento e distribuído em várias instâncias em produção.

2.2 Cliente-servidor

No modelo cliente-servidor, um ou mais clientes solicitam serviços a um componente servidor que controla determinada capacidade ou recurso. É uma arquitetura simples de entender e adequada quando existe uma autoridade natural sobre os dados. O servidor pode centralizar autenticação, regras de negócio e persistência, mas também pode se tornar gargalo ou ponto único de falha se não houver replicação e balanceamento.

- Vantagens: modelo mental simples, controle centralizado, segurança e governança mais diretas.
- Limitações: dependência do servidor, necessidade de dimensionamento, potencial gargalo e ponto único de falha.
- Exemplos: banco de dados, servidor Web, serviço de autenticação, API corporativa.

2.3 Arquitetura multicamadas

A separação em camadas divide apresentação, lógica de negócio e dados. Em uma arquitetura de três camadas, o cliente não acessa diretamente o banco; ele utiliza uma camada de aplicação que

implementa regras e coordena a persistência. Essa separação favorece manutenção e segurança, embora aumente o número de hops e pontos de observabilidade.

2.4 Peer-to-peer (P2P)

Em P2P, os nós podem atuar simultaneamente como clientes e servidores. A descentralização reduz dependência de um coordenador único e permite que capacidade cresça com o número de participantes, mas cria desafios de descoberta, confiança, disponibilidade e consistência. Sistemas de compartilhamento de arquivos e várias blockchains utilizam topologias P2P.

2.5 SOA e microsserviços

Arquiteturas orientadas a serviços dividem um sistema em capacidades acessíveis por interfaces. Microsserviços levam essa ideia a serviços menores e implantáveis independentemente, com forte ênfase em autonomia de equipes e ciclo de implantação. O benefício de evolução independente vem acompanhado de custo distribuído: observabilidade, versionamento de API, dados descentralizados, segurança entre serviços e depuração de fluxos que atravessam múltiplos componentes.

Cuidado com o “microsserviço por padrão”
Distribuir componentes é uma decisão arquitetural, não um objetivo. Para uma equipe pequena e domínio simples, um monólito modular pode entregar melhor confiabilidade e menor custo operacional. A decomposição deve ser justificada por limites de domínio, escala, autonomia de implantação ou outros requisitos concretos.

2.6 Arquiteturas orientadas a eventos

Em arquiteturas orientadas a eventos, produtores publicam fatos ou notificações sem necessariamente conhecer todos os consumidores. Um broker, log distribuído ou barramento roteia as mensagens. O modelo reduz acoplamento espacial e pode reduzir acoplamento temporal, mas exige lidar com duplicação, ordem, retenção, reprocessamento e consistência eventual.

2.7 Middleware

Middleware é a camada de software que oferece abstrações de comunicação e serviços comuns entre aplicações distribuídas. Bibliotecas RPC escondem serialização e transporte; brokers implementam roteamento e filas; service meshes podem tratar segurança e telemetria; frameworks Web abstraem HTTP. Middleware simplifica tarefas repetitivas, mas também introduz semântica própria que precisa ser compreendida.

2.8 Cloud, edge e continuum

Cloud computing concentra capacidade elástica em data centers, enquanto edge computing desloca processamento para perto da origem dos dados. O continuum edge-cloud combina ambos: filtragem e reação local reduzem latência e uso de banda, enquanto armazenamento e análise pesada permanecem na nuvem. O particionamento correto depende de latência, conectividade, custo, privacidade e capacidade do dispositivo.

Tabela 5 - Comparação arquitetural

Arquitetura	Acoplamento	Ponto forte	Custo / risco	Quando considerar
Cliente-servidor	Moderado	Simplicidade e autoridade central	Ponto de falha/gargalo	Serviços com dono claro

Arquitetura	Acoplamento	Ponto forte	Custo / risco	Quando considerar
3 camadas	Moderado	Separação de responsabilidades	Mais hops e operação	Aplicações empresariais
P2P	Baixo em autoridade central	Descentralização	Descoberta e consistência	Compartilhamento/descentralização
Microsserviços	Interfaces explícitas	Evolução e escala independente	Complexidade operacional	Domínios grandes/equipes independentes
Orientada a eventos	Baixo entre produtor/consumidor	Desacoplamento e extensibilidade	Ordem, duplicação, rastreabilidade	Fluxos assíncronos e telemetria

2.9 Estudo de caso: sistema de monitoramento ambiental

Considere sensores que enviam temperatura, um painel Web que mostra o estado atual e um serviço de alertas que identifica limites. Uma solução inicial pode usar um servidor central: sensores enviam dados por HTTP e o painel consulta o banco. Ao adicionar muitos sensores e múltiplos consumidores, um broker de eventos pode desacoplar a ingestão dos consumidores. Um serviço de previsão pode expor uma interface gRPC para processamento de baixa latência. O mesmo domínio permite observar que diferentes mecanismos podem coexistir.

Exercícios de fixação

1. Escolha uma arquitetura para um sistema de matrícula acadêmica e justifique considerando consistência, escala e governança.

2. Explique por que uma arquitetura orientada a eventos pode aumentar a dificuldade de depuração.

3. Um grupo propõe dividir um sistema simples em 25 microsserviços. Liste cinco custos operacionais que devem ser considerados.

4. Compare P2P e cliente-servidor quanto a descoberta, autoridade e tolerância a falhas.

5. Desenhe uma implantação possível para uma arquitetura lógica de três camadas com alta disponibilidade.

2.10 Padrões arquiteturais de apoio

API Gateway

Um API Gateway fornece um ponto de entrada para clientes externos e pode centralizar roteamento, autenticação, limitação de taxa, terminação TLS e composição simples. Ele reduz conhecimento do cliente sobre a topologia interna, mas pode se tornar componente crítico. Regras de negócio complexas no gateway tendem a criar um novo monólito difícil de evoluir.

Service discovery e balanceamento

Quando existem várias réplicas de um serviço, clientes precisam descobrir endereços válidos e distribuir chamadas. Descoberta pode ser resolvida por DNS, registro de serviços, plataforma de orquestração ou proxy. Balanceamento não é apenas round-robin: saúde, afinidade, latência, capacidade e região podem influenciar a decisão.

Orquestração versus coreografia

Em uma orquestração, um componente coordenador determina a sequência de passos: chamar estoque, depois pagamento, depois expedição. Em coreografia, serviços reagem a eventos sem um controlador central explícito. Orquestração facilita visualizar o fluxo e compensações; coreografia reduz acoplamento direto, mas pode espalhar a lógica de processo e dificultar rastreabilidade. Sistemas reais frequentemente combinam ambos.

Bulkhead e isolamento

O padrão bulkhead separa recursos para evitar que saturação de uma função derrube outras. Pools de threads, filas, limites de conexão e quotas podem ser isolados. A ideia vem de compartimentos de navios: uma infiltração local não deve alagar toda a embarcação. Em microsserviços, isolar chamadas lentas impede exaustão global de workers.

2.11 Escala horizontal, vertical e particionamento

Escala vertical aumenta recursos de uma instância; escala horizontal adiciona instâncias. Horizontal costuma exigir que estado de sessão não fique preso a um único processo ou que exista estratégia explícita de afinidade. Quando o volume de dados ultrapassa uma única instância, particionamento (sharding) divide dados por chave. A escolha da chave influencia distribuição de carga, consultas e possibilidade de hotspots.

Replicação e particionamento resolvem problemas diferentes. Replicação cria cópias do mesmo dado para disponibilidade ou leitura; particionamento divide o conjunto para capacidade. É comum combinar ambos: cada partição possui múltiplas réplicas. Essa combinação melhora escala, mas aumenta o problema de coordenação e recuperação.

2.12 Monólito modular e sistemas distribuídos

Uma disciplina de Sistemas Distribuídos não deve ensinar que toda aplicação deve ser distribuída. Um monólito modular preserva chamadas locais e transações simples, reduzindo modos de falha. A distribuição deve surgir quando há requisitos como escala independente, isolamento, integração de tecnologias heterogêneas, implantação autônoma, proximidade física com dispositivos ou fronteiras organizacionais. Migrar cedo demais para microsserviços pode transformar problemas de modularidade em problemas de rede.

3. Paradigmas de Comunicação em Sistemas Distribuídos

Objetivos de aprendizagem

- Distinguir comunicação baseada em recursos, eventos e chamadas remotas.
- Explicar acoplamento espacial, temporal e de interface.
- Relacionar sincronismo, bloqueio, buffering e backpressure.
- Escolher mecanismos de comunicação de acordo com requisitos.

3.1 Mensagens atravessam fronteiras

Processos distintos não compartilham diretamente pilha, heap ou referências de objetos. Para cooperar, precisam transformar dados em uma representação transportável, enviá-la por um canal e reconstruí-la no destino. Esse processo envolve serialização, endereçamento, transporte, controle de erros e, frequentemente, autenticação.

3.2 Sincronismo e assincronismo

Em uma interação síncrona típica, o chamador aguarda uma resposta para continuar. Em uma interação assíncrona, o emissor pode prosseguir após entregar uma mensagem a um intermediário ou buffer. Isso não significa que protocolos assíncronos sejam sempre mais rápidos; significa que a dependência temporal entre os participantes é diferente.

3.3 Três formas de acoplamento

- Acoplamento espacial: o emissor precisa conhecer a identidade/endereço do receptor?
- Acoplamento temporal: os dois lados precisam estar disponíveis ao mesmo tempo?
- Acoplamento de interface: o emissor depende fortemente da assinatura e do esquema de operações do receptor?

3.4 Recursos, eventos e chamadas remotas

Tabela 6 - Comparação inicial dos paradigmas de comunicação

Dimensão	REST / recursos	MQTT / eventos	gRPC / RPC
Abstração principal	Recurso identificado por URI	Mensagem publicada em tópico	Método de serviço remoto
Interação típica	Request/response	Publish/subscribe	Chamada/retorno
Acoplamento espacial	Cliente conhece endpoint	Produtor conhece broker/tópico, não consumidores	Cliente conhece endpoint e contrato
Acoplamento temporal	Normalmente alto	Pode ser reduzido por broker/sessão	Normalmente alto
Interface	HTTP + representação	Tópicos + payload	IDL fortemente definida
Serialização comum	JSON	Payload livre/JSON/binário	Protocol Buffers
Uso típico	APIs públicas e CRUD	Telemetria e eventos	Serviço-a-serviço de baixa latência

3.5 Latência, throughput e backpressure

Latência mede o tempo para concluir uma operação ou atravessar um caminho; throughput mede quantidade de trabalho por unidade de tempo. Um sistema pode ter excelente throughput e latência ruim, ou o inverso. Backpressure aparece quando produtores geram dados mais rapidamente do que consumidores conseguem processar. Filas crescentes apenas adiam o problema; arquiteturas maduras definem limites, políticas de descarte, controle de fluxo ou redução de produção.

3.6 Idempotência e retries

Retry é uma resposta comum a falhas transitórias, mas pode duplicar efeitos. Se um cliente envia “debitar R$ 100”, o servidor processa e a resposta se perde, uma repetição ingênua pode debitar novamente. Operações idempotentes produzem o mesmo efeito observável quando repetidas com os mesmos parâmetros. HTTP define métodos com semânticas distintas, e aplicações podem usar chaves de idempotência para proteger operações não idempotentes.

Regra de engenharia
Nunca configure retries sem definir: quais erros são transitórios, qual o limite de tentativas, qual estratégia de backoff, como evitar tempestades de retry e qual a semântica de duplicação da operação.

Exercícios de fixação

1. Explique por que assíncrono não significa “sem resposta” e síncrono não significa “uso obrigatório de HTTP”.

2. Dê um exemplo de backpressure em um sistema de telemetria e proponha duas estratégias de controle.

3. Classifique REST, MQTT e gRPC segundo acoplamento espacial e temporal em um cenário concreto.

4. Por que retries são particularmente perigosos em operações com efeitos financeiros?

5. Proponha um mecanismo de correlação entre requisição e resposta em uma arquitetura baseada em mensagens.

AP8 - Contrato inteligente e integração local

Atividade prática
Desenvolva, implante e teste um contrato inteligente que mantenha estado e emita eventos. A atividade deve modificar significativamente o exemplo e incluir pelo menos um requisito de acesso ou validação.

Objetivos da atividade

- Modelar estado on-chain.
- Implementar funções de leitura e escrita.
- Emitir e observar eventos.
- Criar testes automatizados.
- Analisar custo, segurança e integração off-chain.

Requisitos obrigatórios

- Contrato diferente do exemplo.
- Pelo menos duas funções que alteram estado.
- Pelo menos uma função view.
- Ao menos um evento.
- Uma regra de require/revert significativa.
- No mínimo quatro testes automatizados, incluindo falha esperada.

Roteiro mínimo

1. Defina o estado e invariantes.
2. Implemente o contrato.
3. Compile e execute testes.
4. Implante em nó local.
5. Execute transações e consulte estado.

6. Observe eventos.
7. Introduza tentativa inválida e confirme revert.

Entregáveis

- Contrato e testes.
- Comandos de compilação/implantação.
- Hash/endereço local da implantação como evidência de laboratório.
- Logs/eventos observados.
- Análise de pelo menos três riscos de segurança.
- Comparação entre lógica on-chain e serviço REST tradicional.

Questões para análise

1. Que parte do estado precisa realmente estar on-chain?
2. Qual função consome gas quando enviada como transação e por quê?
3. Como eventos ajudam sistemas off-chain?
4. Que vulnerabilidade seria possível se controle de acesso estivesse ausente?
5. Quando um banco de dados tradicional seria escolha superior?

Critérios de avaliação

Tabela 22 - Critérios de avaliação da AP8

Critério	Evidência esperada
Modelagem	Estado e regras coerentes.
Contrato	Compila e executa corretamente.
Testes	Cobrem sucesso e falha.
Segurança	Riscos analisados e mitigados quando possível.
Integração	Eventos e consultas demonstrados.

Checklist antes da apresentação

☐ Contrato compila.

☐ Há teste de revert.

☐ Eventos foram observados.

☐ Implantação é em ambiente sem valor real.

☐ A análise discute necessidade de blockchain.

Desafios opcionais

- Crie frontend simples.
- Implemente papéis de acesso.
- Meça gas de diferentes operações.

14.9 Reentrância, chamadas externas e ordem de efeitos

Reentrância ocorre quando um contrato faz chamada externa antes de concluir suas próprias atualizações e o contrato chamado volta a invocar o primeiro em estado intermediário. Uma defesa clássica é checks-effects-interactions: validar condições, atualizar estado interno e somente depois interagir externamente. Reentrancy guards podem complementar o padrão. O ponto distribuído é que uma chamada externa amplia a superfície de interação e reduz controle local sobre o fluxo.

14.10 Controle de acesso

Usar msg.sender sem regra clara não basta. Contratos podem precisar de owner, papéis ou listas de permissões. Toda função que altera estado sensível deve ter requisito de autorização explícito. Testes devem provar não só que o usuário autorizado consegue executar, mas que um endereço não autorizado recebe revert.

14.11 Oráculos e fronteira on-chain/off-chain

A EVM não pode simplesmente consultar uma API Web durante execução determinística, pois diferentes validadores poderiam obter respostas distintas. Oráculos são mecanismos para introduzir dados externos segundo regras específicas. Isso mostra um limite fundamental: consenso on-chain não torna a fonte externa verdadeira. A confiança é deslocada para desenho do oráculo, agregação e incentivos.

14.12 Upgradeability e governança

Contratos imutáveis simplificam raciocínio, mas dificultam correção de bugs. Padrões de proxy permitem atualizar lógica mantendo estado, porém introduzem complexidade e uma autoridade de

upgrade. A pergunta muda de “o código é imutável?” para “quem pode mudar o código, sob quais regras e com que transparência?”. Em sistemas distribuídos, governança faz parte do modelo de confiança.